

Backtesting — ทดสอบก่อนใช้เงินจริง

บทที่ 22–25: Vectorized · Event-Driven · Walk-Forward · Slippage & Fees

จบ Part นี้ → รู้ว่า strategy ที่ดูดีบนกระดาษ จะรอดในชีวิตจริงไหม

🔗 อ่าน Part นี้ยังไ้

● **มาถึงตรงนี้ได้แล้ว** — Part นี้สำคัญกว่าที่คิด: มันคือส่วนที่ตัดสินใจว่ากลยุทธ์ของคุณจริงหรือหลอกตัวเอง · ถ้าเวลาน้อยให้อ่าน **22.1 (lookahead bias)** กับ **บทที่ 25 (ต้นทุน)** ให้เข้าใจก่อน — สองเรื่องนี้ทำให้ backtest ส่วนใหญ่ในอินเทอร์เน็ตใช้ไม่ได้ · และดูตัวเลขติดลบในบทนี้ให้ดี มันตั้งใจให้ติดลบ

● **เคยเขียน backtest มาแล้ว** — ข้ามโครงสร้างพื้นฐานได้ · ที่ควรอ่าน: **22.1b (full-sample statistics)** ซึ่งเป็น lookahead ที่แนบเนียนกว่าการ shift(1) มาก · **บทที่ 24** ที่การแบ่ง train/test ผิดทำให้ "out-of-sample" ไม่ใช่ out-of-sample จริง · และตาราง cost sensitivity ในบทที่ 25 ที่ Sharpe ไหลจาก 2.67 → 0.21 ด้วยกลยุทธ์เดียวกันเป๊ะ โดย **ไม่ติดลบสักบรรทัด**

ทำไม BACKTESTING ถึงสำคัญมาก?

ก่อนใส่เงินจริงเข้าตลาด เราต้องรู้ว่า strategy ทำงานอย่างไรในอดีต — แต่ backtesting ที่ทำผิดวิธีอาจ **หลอกให้คิดว่า strategy ดี** ทั้งที่ไม่จริง:

- **Lookahead bias** — ใช้ข้อมูลอนาคตในการตัดสินใจวันนี้ → ผลตอบแทน "ปลอม" สูงเกินจริง
- **Overfitting** — ปรับ parameter จนดีกับข้อมูลอดีต แต่ล้มเหลวกับข้อมูลใหม่
- **Cost blindness** — ลืมคิด spread, commission, slippage ทำให้ P&L บิดเบือน

Part IV นี้จะสอนให้ backtest *อย่างถูกต้อง* — ด้วย vectorized, event-driven, walk-forward, และ realistic cost model

Running Example ตลอด Part IV — BTC Bybit/Binance Pair Strategy

เราจะต่อยอด PairStrategy และ TradingSystem จาก Part II/III มาทดสอบอย่างเป็นระบบ:

บท 22 → Vectorized backtest เร็ว · บท 23 → Event-driven สมจริง

บท 24 → Walk-forward ป้องกัน overfit · บท 25 → ใส่ต้นทุนจริง และดูว่า edge ยังอยู่ไหม

บทที่ 22: Vectorized Backtest

แก่นของบทนี้

Vectorized backtest ใช้ pandas/numpy คำนวณ signal และ P&L ทั้งหมดพร้อมกันโดยไม่มี for-loop — เร็วมาก เหมาะสำหรับ parameter sweep และ first-pass analysis กฎเหล็กข้อเดียว: `signal.shift(1)` ก่อนคุณ return เสมอ เพื่อหลีกเลี่ยง lookahead bias

แบบทดสอบก่อนเริ่ม — Lookahead Bias คืออะไร?

สมมติว่าคุณเขียนโค้ดแบบนี้:

```
signal = (zscore > 2.0).astype(int)
ret_strategy = signal * daily_return
```

? โค้ดนี้มี lookahead bias หรือเปล่า?

คำตอบ: มี — `signal[t]` คำนวณจาก zscore วัน `t` แต่ `daily_return[t]` = ราคาปิดวัน `t` / ราคาปิดวัน `t-1` ซึ่งคุณยังไม่รู้ตอน `t` เริ่มต้น
แก้ด้วย: `ret_strategy = signal.shift(1) * daily_return`
กฎเหล็ก: signal วัน `t` → execute วัน `t+1` เสมอ

22.1 Lookahead Bias — ศัตรูตัวร้ายของ Backtesting

Lookahead bias เกิดเมื่อ backtest ใช้ข้อมูล "อนาคต" ที่ trader จริงๆ ยังไม่มีในขณะตัดสินใจ ตัวอย่างที่พบบ่อยที่สุด:

```
# ตัวอย่างข้อมูลจำลอง BTC Bybit/Binance import numpy as np import pandas as pd np.random.seed(42) n = 500 dates =
pd.date_range("2023-01-01", periods=n, freq="D") bybit = 25000 + np.cumsum(np.random.normal(20, 300, n)) # ส่วน
ต่างสองตลาดต้อง "ติงกลับ" ได้ ไม่ใช่สุ่มลอย ๆ - ใช้ OU เหมือนบทที่ 12 # ถ้าใช้ noise เจย ๆ spread จะไม่มีแรงติงกลับ = กลยุทธ์นี้ไม่มี edge
ตั้งแต่ต้น kappa, sigma_ou = 0.15, 140.0 ou = np.zeros(n) for t in range(1, n): ou[t] = ou[t-1] + kappa * (0 - ou[t-
1]) + np.random.normal(0, sigma_ou) binance = bybit + ou # ราคาใกล้เคียง Bybit + spread ที่ mean-revert df =
pd.DataFrame({"bybit": bybit, "binance": binance}, index=dates) # คำนวณ z-score ของ spread WINDOW = 60 spread =
np.log(df["bybit"]) - np.log(df["binance"]) roll_mean = spread.rolling(WINDOW).mean() roll_std =
spread.rolling(WINDOW).std() zscore = (spread - roll_mean) / roll_std ENTRY_Z, EXIT_Z = 2.0, 0.5 # Raw signal
(ยังไม่ lag) raw_signal = pd.Series(0.0, index=df.index) raw_signal[zscore > ENTRY_Z] = -1.0 # short spread
raw_signal[zscore < -ENTRY_Z] = 1.0 # long spread # forward-fill: ถือ position ต่อจนกว่า signal จะเปลี่ยน raw_signal
= raw_signal.replace(0, np.nan).ffill().fillna(0) #

# คำนวณ daily return ของ spread spread_return
= df["bybit"].pct_change() - df["binance"].pct_change() # ❌ ผิด - Lookahead Bias: signal วัน t คุณ return วัน t #
ราคา close ที่คำนวณ signal กับราคา que execute เป็นวันเดียวกัน bad_pnl = (raw_signal * spread_return).dropna() # ✅ ถูก -
shift(1): signal วัน t execute ที่ราคาวัน t+1 good_signal = raw_signal.shift(1).fillna(0) good_pnl =
(good_signal * spread_return).dropna() print(f"Same-bar (ไม่ shift) cumulative return: {(1+bad_pnl
).prod()-1:+.2%}") print(f"Shift(1) (ถูกต้อง) cumulative return: {(1+good_pnl).prod()-1:+.2%}") print(f"ส่วนต่าง:
{(1+bad_pnl).prod()-(1+good_pnl).prod():+.2%}")
```

► OUTPUT

```
Same-bar (ไม่ shift) cumulative return: +7.11%
Shift(1) (ถูกต้อง) cumulative return: +17.65%
ส่วนต่าง: -10.55%
```

😓 เอ๊ะ — ทำไมเวอร์ชัน "ผิด" กลับได้น้อยกว่า?

ผลออกมาสวนความคาดหมาย และนี่คือบทเรียนที่สำคัญกว่าตัวเลข · เหตุผล: signal ของเราเป็น **mean-reversion** — `raw_signal[t]` ถูกจุด
ชนวนโดยการเคลื่อนไหวที่เพิ่งเกิดขึ้นในวัน `t` เอง · พอเอาไปคูณ `spread_return[t]` ซึ่งคือการเคลื่อนไหวอันเดียวกันนั้น เราจึงกำลัง "เติมพันสวน

ทางหลังจากมันวิ่งไปแล้ว" = ขาดทุนที่แท่ง entry ทุกครั้ง

🚫 **สิ่งที่ต้องจำ:** ไม่ใช่ทุก lookahead ที่ทำให้ผลดูดีขึ้น — แต่ทุก lookahead ทำให้ผล **ไม่จริง** · อย่าใช้ "ผลดูดีเกินไป" เป็นเครื่องตรวจจับ lookahead เพราะบางแบบทำให้ผลดูแย่ลง แล้วคุณก็จะทิ้งกลยุทธ์ที่ดีไปโดยไม่รู้ตัว · เครื่องตรวจจับที่เชื่อถือได้มีอย่างเดียว: **ไล่ดูทุกบรรทัดว่าข้อมูลที่ใช้นั้น ณ เวลา t มีอยู่จริงตอน t หรือยัง**

22.1b Lookahead แบบที่มองด้วยตาแทบไม่เห็น — full-sample statistics

แบบที่ค่อนข้างเก่งกว่ามาก เพราะมันไม่ได้อยู่ในบรรทัดที่เกี่ยวกับ **เวลา** เลย: ค่า mean/std ของ z-score จาก **ข้อมูลทั้งชุด** แทนที่จะเป็น rolling window · โค้ดสั้นกว่า อ่านสะอาดกว่า และไม่มีอะไรในหน้าตาของมันบอกว่าคุณ

```
# ❌ ผิด — ใช้ mean/std ของทั้งชุด: วันแรกของ backtest "รู้" ค่าเฉลี่ยของทั้งปีแล้ว z_full = (spread - spread.mean()) / spread.std() # ✅ ถูก — rolling: ใช้เฉพาะข้อมูลที่มีจริง ณ เวลานั้น z_roll = (spread - spread.rolling(WINDOW).mean()) / spread.rolling(WINDOW).std() def make_signal(z): # NaN = "ไม่มีเหตุการณ์" · ตัวเลข = "มีเหตุการณ์" (เข้า หรือ ออก) s = pd.Series(np.nan, index=df.index) s[z > ENTRY_Z] = -1.0 s[z < -ENTRY_Z] = 1.0 s[z.abs() < EXIT_Z] = 0.0 # exit ก็เป็นเหตุการณ์เหมือนกัน return s.ffill().fillna(0).shift(1).fillna(0) pnl_full = (make_signal(z_full) * spread_return).dropna() pnl_roll = (make_signal(z_roll) * spread_return).dropna() print(f"❌ full-sample z: {(1+pnl_full).prod()-1:+.2%}") print(f"✅ rolling z: {(1+pnl_roll).prod()-1:+.2%}") print(f"ghost profit: {(1+pnl_full).prod()-(1+pnl_roll).prod():+.2%}")
```

► OUTPUT

```
❌ full-sample z: +19.35%
✅ rolling z:      +19.78%
ghost profit:    -0.43%
```

🔴 **อ่านว่า:** spread.mean() ดูไร้พิษภัยมาก — แต่มันคือค่าเฉลี่ยของทั้ง **500 วัน** · การใช้มันตัดสินใจในวันที่ 61 เท่ากับบอกว่า "ตอนนั้นฉันรู้แล้วว่าอีก 439 วันข้างหน้าราคาเฉลี่ยจะเป็นเท่าไร" · ต่างจาก .rolling(60).mean() ที่มองย้อนหลังแค่ 60 วันที่ผ่านมาแล้วจริง ๆ

😞 **อ้าว — ตัวเลขบอกว่ารู้แล้วได้น้อยลง 0.43%**

ถ้าคุณกำลังรอให้ตัวเลข **❌** สูงกว่า **✅** เพื่อจับผิด — มันไม่มาแบบนั้น · และนี่คือเหตุผลที่หัวข้อนี้นี้อยู่

ข้อมูลชุดเดียวบอกอะไรไม่ได้เลย เพราะส่วนต่างนี้เป็นเรื่องของ **โชคในชุดข้อมูลนั้น** · วิธีเดียวที่จะรู้ว่าการรู้ล่วงหน้าส่งผลอย่างไร คือทำซ้ำหลาย ๆ ชุดแล้วดูการกระจาย ไม่ใช่ชุดเดียว

ลองสร้างข้อมูลแบบเดียวกัน 300 ชุด (คนละเมล็ดสุ่ม) แล้ววัดส่วนต่างทุกชุด:

```
# ทำซ้ำ 300 ชุด — ดูว่า "การรู้" ส่งผลอย่างไรโดยรวม ไม่ใช่ดูชุดเดียว def leak_gap(rng, n=500): # สร้างข้อมูล 1 ชุด แล้วคืน (ผลของ z แบบรู้) ลบ (ผลของ z แบบถูก) bybit = 25000 + np.cumsum(rng.normal(20, 300, n)) ou = np.zeros(n) for t in range(1, n): ou[t] = ou[t-1] + 0.15 * (0 - ou[t-1]) + rng.normal(0, 140.0) idx = pd.date_range("2023-01-01", periods=n, freq="D") d = pd.DataFrame({"bybit": bybit, "binance": bybit + ou}, index=idx) sp = np.log(d["bybit"]) - np.log(d["binance"]) zf = (sp - sp.mean()) / sp.std() # รู้ zr = (sp - sp.rolling(WINDOW).mean()) / sp.rolling(WINDOW).std() r = d["bybit"].pct_change() - d["binance"].pct_change() def sig(z): s = pd.Series(np.nan, index=idx) s[z > ENTRY_Z] = -1.0 s[z < -ENTRY_Z] = 1.0 s[z.abs() < EXIT_Z] = 0.0 return s.ffill().fillna(0).shift(1).fillna(0) return ((1 + (sig(zf) * r).dropna()).prod() - (1 + (sig(zr) * r).dropna()).prod()) rng = np.random.default_rng(0) gaps = np.array([leak_gap(rng) for _ in range(300)]) print("ทดลอง 300 ชุดข้อมูล — ส่วนต่าง (z แบบรู้ ลบ z แบบถูก)") print(f"เฉลี่ย : {gaps.mean():+.2%}") print(f"ส่วนเบี่ยงเบน : {gaps.std():.2%}") print(f"แบบรู้ชนะ : {(gaps > 0).mean():.0%} ของชุดข้อมูล") print(f"ช่วง 5%-95% : {np.percentile(gaps, 5):+.2%} " f"ถึง {np.percentile(gaps, 95):+.2%}")
```

► OUTPUT

```
ทดลอง 300 ชุดข้อมูล — ส่วนต่าง (z แบบรู้ ลบ z แบบถูก)
เฉลี่ย          : +0.73%
ส่วนเบี่ยงเบน   : 5.06%
แบบรู้ชนะ      : 55% ของชุดข้อมูล
ช่วง 5%-95%     : -7.92% ถึง +9.05%
```

อ่านตัวเลขสื่อบรรทัดนี้ให้ขาด — มันเปลี่ยนวิธีคิดเรื่อง lookahead ทั้งหมด

ค่าเฉลี่ย +0.73% แทบไม่ใช้ประเด็น · ประเด็นอยู่ที่ ส่วนเบี่ยงเบน 5.06% และช่วง -7.92% ถึง +9.05%

แปลเป็นภาษาคน: การรู้แบบนี้ไม่ได้ทำให้ผลดีขึ้นอย่างเป็นระบบ — มันทำให้ผลมั่วอย่างเป็นระบบ · backtest ชุดใดชุดหนึ่งอาจเกินจริงไป 9% หรือต่ำกว่าจริง 8% และคุณไม่มีทางรู้ว่าชุดของคุณอยู่ตรงไหน เพราะคุณมีข้อมูลชุดเดียว

ถ้าจับ lookahead ด้วยวิธี "ดูว่าผลดีเกินจริงไหม" คุณจะพลาดทันที 45% ของกรณี (ชุดที่รู้แล้วผลออกมาแย่ลง) · และในชุดที่ผลออกมาดีขึ้น ก็ยังแยกไม่ออกอยู่ดีว่ามาจาก edge จริงหรือจากการรู้

❌ ข้อสรุปที่ใช้ได้ตลอด: อย่าถามว่า "การรู้แบบนี้ทำให้ผลดีขึ้นเท่าไร" — ให้ถามว่า "ผลนี้ทำซ้ำในชีวิตจริงได้ไหม" · `spread.mean()` ของทั้งชุดไม่มีอยู่จริงในวันที่ 61 ดังนั้น backtest ที่ใช้มันจึงเป็นตัวเลขที่ไม่มีใครทำซ้ำได้ — จบตรงนั้น ไม่ต้องเถียงกันว่ามันบวกหรือลบ

● [รอบสอง] ทำไมการรู้แบบนี้ถึงไม่กำไรอย่างที่เราคิด

สัญญาณญาณบอกว่า "รู้อนาคต = ได้เปรียบ" แต่ `spread.mean()` ของทั้งชุดไม่ใช่ข้อมูลอนาคตที่เอาไปทำเงินได้โดยตรง · มันเป็นแค่ค่าคงที่ตัวหนึ่งที่ไปเลื่อนตำแหน่งเส้นแบ่ง entry/exit ทั้งเส้น

ผลคือกลยุทธ์เข้าเทรดคนละจุดกันไปเลยทั้งชุด — ไม่ใช่ "เข้าจุดเดิมแต่ได้เปรียบขึ้น" · จุดใหม่นั้นบางชุดบังเอิญดีกว่า บางชุดบังเอิญแยกว่า จึงออกมาเป็น 55/45

เทียบกับการรู้ที่ให้กำไรจริง — เช่นเอาราคาพรงนี้มาตัดสินใจวันนี้ (`shift(-1)`) · แบบนั้นกลยุทธ์จะชนะเกือบทุกชุดข้อมูล และ Sharpe จะพุ่งไปในระดับที่เป็นไปไม่ได้ · เส้นแบ่งอยู่ที่ว่าสิ่งที่รู้ออกมาบอก "ทิศทางของราคาข้างหน้า" หรือแค่ "บริบททางสถิติ"

สองอย่างนี้ต้องแก้เหมือนกัน แต่อาการต่างกันคนละขั้ว — และเพราะแบบหลังไม่มีการให้เห็น มันจึงรอดสายตาไปถึง production ได้บ่อยกว่ามาก

กฎเหล็ก 2 ข้อของ Vectorized Backtest

① `shift(1)` เสมอ — signal สร้างจากราคา close วันที่ $t \rightarrow$ execute ได้เร็วสุดที่ open วันที่ $t + 1$

② ทุกสถิติต้องเป็น rolling หรือ expanding เท่านั้น — เจอ `.mean()`, `.std()`, `.min()`, `.max()`, `.quantile()` ที่ไม่มี `.rolling()` หรือ `.expanding()` นำหน้า ให้สงสัยไว้ก่อนทุกครั้ง · รวมถึงการ normalize/scale ข้อมูลก่อนแบ่ง train-test ด้วย (เรื่องนี้ลึกใน Part V)

22.2 Vectorized Backtest ฉบับสมบูรณ์

📖อ่านว่า: ฟังก์ชันนี้ยาว แต่มีหมายเลขกำกับให้แล้วทั้ง 7 ขั้น — และลำดับของมันไม่ใช่เรื่องบังเอิญ มันคือสายพานเดียวที่แปลงราคาเป็นเงิน: ราคา $\rightarrow$ ① spread/z-score $\rightarrow$ ② สัญญาณ $\rightarrow$ ③ หน่วง 1 แท่ง $\rightarrow$ ④ ผลตอบแทน $\rightarrow$ ⑤ หักค่าธรรมเนียม $\rightarrow$ ⑥ equity $\rightarrow$ ⑦ drawdown · ทุกขั้นเป็นคอลัมน์ทั้งคอลัมน์ ไม่มีลูปสักตัว นั่นคือความหมายของคำว่า vectorized · และเหตุผลที่มันเร็วกว่าการวนทีละแท่งเป็นร้อยเท่า คือ pandas ส่งงานทั้งคอลัมน์ลงไปให้โค้ดภาษา C คิดรวดเดียว

```
def vectorized_backtest( price_a: pd.Series, price_b: pd.Series, window: int = 60, entry_z: float = 2.0, exit_z: float = 0.5, fee_rate: float = 0.001, ) -> pd.DataFrame: """ Vectorized pair trading backtest un price series A และ B ส่งคืน DataFrame ที่มี: signal, net_return, equity, drawdown """ df = pd.DataFrame({"A": price_a, "B": price_b}) # 1. Spread และ z-score _____ log_spread = np.log(df["A"]) - np.log(df["B"]) roll_mean = log_spread.rolling(window).mean() roll_std = log_spread.rolling(window).std() df["zscore"] = (log_spread - roll_mean) / roll_std # 2. Raw signal (ยังไม่ shift) _____ # NaN = "แท่งนี้ไม่มีอะไรเกิดขึ้น" -> ffill จะถือ position เดิมต่อ # ตัวเลข = "มีเหตุการณ์" ซึ่งมี 3 แบบ: เข้า short · เข้า long · ออก raw = pd.Series(np.nan, index=df.index) raw[df["zscore"] > entry_z] = -1.0 # spread กว้างเกิน -> short raw[df["zscore"] < -entry_z] = 1.0 # spread แคบเกิน -> long raw[df["zscore"].abs() < exit_z] = 0.0 # กลับสู่ปกติ -> ปิด # ⚠ ffill ต้องมา "หลัง" ทั้งเข้าและออกถูก mark ครบแล้วเท่านั้น raw = raw.ffill().fillna(0) # 3. SHIFT(1) -> หัวใจสำคัญ _____ df["signal"] = raw.shift(1).fillna(0) # 4. Returns _____ ret_a = df["A"].pct_change() ret_b = df["B"].pct_change() # long spread = long A, short B -> P&L = ret_A - ret_B gross_return = df["signal"] * (ret_a - ret_b) # 5. Transaction cost (เมื่อ position เปลี่ยน) _____ position_change = df["signal"].diff().abs() # 1 เมื่อ flip/exit cost_per_bar = position_change * fee_rate * 2 # ทั้ง A และ B df["net_return"] = gross_return - cost_per_bar #
```

```
6. Equity curve (normalized: เริ่มที่ 1.0) ————— df["equity"] = (1 + df["net_return"]).cumprod() # — 7.
Drawdown ————— peak = df["equity"].cummax() df["drawdown"] =
(df["equity"] - peak) / peak return df.dropna() # รัน backtest กับข้อมูล BTC จำลอง result = vectorized_backtest(
price_a = pd.Series(bybit, index=dates, name="Bybit"), price_b = pd.Series(binace, index=dates,
name="Binace"), window=60, entry_z=2.0, exit_z=0.5, fee_rate=0.001, ) print(result[["zscore", "signal",
"net_return", "equity", "drawdown"]].tail(6).round(5))
```

► OUTPUT

```
zscore signal net_return equity drawdown
2024-05-09 0.95021 0.0 0.0 1.13252 -0.002
2024-05-10 1.24011 0.0 0.0 1.13252 -0.002
2024-05-11 1.15657 0.0 -0.0 1.13252 -0.002
2024-05-12 0.11836 0.0 -0.0 1.13252 -0.002
2024-05-13 -0.21545 0.0 -0.0 1.13252 -0.002
2024-05-14 0.07878 0.0 0.0 1.13252 -0.002
```

🚩 3 บรรทัดที่มีมือใหม่มักเขียนผิด และวิธีอ่านให้ถูก

```
raw = pd.Series(np.nan, index=df.index) ① NaN = "ไม่มีเหตุการณ์"
raw[z > entry_z] = -1.0
raw[z < -entry_z] = 1.0
raw[z.abs() < exit_z] = 0.0 exit ก็เป็นเหตุการณ์
raw = raw.ffill().fillna(0) ffill ที่เดียวตอนท้าย

position_change = df["signal"].diff().abs() ② นับจังหวะที่ต้องจ่าย fee
cost_per_bar = position_change * fee_rate * 2

df["equity"] = (1 + df["net_return"]).cumprod() ③ ทบต้น ไม่ใช่บวกสะสม
```

1. ทำไมต้องเริ่มด้วย `np.nan` ไม่ใช่ `0.0` — เพราะ `ffill` แปลว่า "ช่องว่างให้เอาค่าก่อนหน้ามาเติม" · ถ้าเริ่มด้วย 0 แล้วค่อยแปลง 0 เป็น NaN ที่หลัง จะเกิดปัญหาว่า 0 ที่แปลว่า "ปิดสถานะ" กับ 0 ที่แปลว่า "ไม่มีอะไรเกิดขึ้น" หน้าตาเหมือนกันจนแยกไม่ออก · การเริ่มจาก NaN ทำให้สองอย่างนี้ ต่างกันตั้งแต่แรก: NaN = เจียบ · 0 = สั่งปิด
2. `diff().abs()` คือจำนวน "ขา" ที่ต้องซื้อขาย — จาก 0→1 ได้ 1 · จาก 1→-1 ได้ 2 (ปิดของเดิมแล้วเปิดฝั่งตรงข้าม จ่ายสองต่อ) · คูณ * 2 ท้ายสุด เพราะ pair trade ขยับสองเหรียญพร้อมกันเสมอ · จับให้ได้ว่า `fee_rate * 2` คือ "สองเหรียญ" ส่วน `diff().abs()` คือ "กี่ครั้ง" — คนละมิติกัน
3. `(1 + r).cumprod()` ไม่ใช่ `r.cumsum()` — เงินทบต้นไม่ได้บวกกันตรง ๆ · ได้ 10% แล้วเสีย 10% ไม่ได้กลับมาที่เดิม แต่เหลือ 0.99 · ในบทที่ 30 จะเห็นอีกสำนวนคือ `np.exp(log_ret.cumsum())` ซึ่งให้ผลเดียวกันเมื่อใช้ผลตอบแทนแบบ log — เลือกอันไหนก็ได้ แต่ห้ามผสมกัน

บรรทัด `return df.dropna()` ตัดแถวแรก ๆ ที่ z-score ยังเป็น NaN ทั้ง (60 แถวแรก) · คูเล็กน้อยแต่สำคัญ — ถ้าไม่ตัด `equity` จะเริ่มนับจากช่วงที่ยังไม่มีสัญญาณ แล้ว `annualized return` จะถูกเจือจางด้วยวันที่ไม่ได้เทรด

22.3 Performance Metrics: Sharpe, Calmar, Max Drawdown

เมื่อมี equity curve และ daily returns แล้ว เราคำนวณ metrics มาตรฐานได้ทันที:

```
def compute_metrics(result: pd.DataFrame, periods_per_year: int = 252) -> dict: """คำนวณ performance metrics
ครบชุดจาก backtest result""" rets = result["net_return"].dropna() eq = result["equity"].dropna() dd =
result["drawdown"].dropna() # Annualized return (compound) total_return = eq.iloc[-1] - 1.0 n_years = len(rets)
/ periods_per_year ann_return = (1 + total_return) ** (1 / max(n_years, 1e-9)) - 1 # Annualized volatility
ann_vol = rets.std() * np.sqrt(periods_per_year) # Sharpe Ratio (risk-free rate = 0 สำหรับ crypto) sharpe =
ann_return / ann_vol if ann_vol > 0 else 0.0 # Max Drawdown max_dd = float(dd.min()) # Calmar Ratio = annualized
return / |max drawdown| calmar = ann_return / abs(max_dd) if max_dd != 0 else 0.0 # Win Rate (บน active days
เท่านั้น) active = rets[rets != 0] win_rate = float((active > 0).mean()) if len(active) > 0 else 0.0 # Profit
Factor = gross profit / gross loss wins = active[active > 0].sum() losses = abs(active[active < 0].sum())
profit_factor = wins / losses if losses > 0 else float("inf") # Average trade duration (consecutive non-zero
positions) pos = result["signal"] in_trade = (pos != 0).astype(int) avg_duration = (in_trade.groupbyby( in_trade
!= in_trade.shift()).cumsum() ).sum().replace(0, np.nan).dropna().mean() return { "Total Return (%)" :
round(total_return * 100, 2), "Ann. Return (%)" : round(ann_return * 100, 2), "Ann. Volatility (%)" :
round(ann_vol * 100, 2), "Sharpe Ratio" : round(sharpe, 3), "Max Drawdown (%)" : round(max_dd * 100, 2), "Calmar
```

```
Ratio" : round(calmar, 3), "Win Rate (%)" : round(win_rate * 100, 1), "Profit Factor" : round(profit_factor, 3),
"Avg Trade Duration" : round(float(avg_duration), 1), } m = compute_metrics(result) print("=" * 44) for k, v in
m.items(): print(f" {k:24s}: {v}")
```

► OUTPUT

```
=====
Total Return (%)      : 13.25
Ann. Return (%)       : 7.37
Ann. Volatility (%)   : 4.15
Sharpe Ratio          : 1.775
Max Drawdown (%)      : -2.24
Calmar Ratio          : 3.296
Win Rate (%)          : 52.9
Profit Factor          : 1.72
Avg Trade Duration    : 7.6
```

ตีความตัวเลข

Sharpe **1.775** · Calmar **3.296** · Profit Factor **1.72** — ตัวเลขสวยมาก และนั่นคือสิ่งที่ต้องระวังที่สุด

สิ่งที่ตัวเลขชุดนี้พิสูจน์คือ **โค้ด backtest ทำงานถูกต้อง** — ไม่ใช่ว่ากลยุทธ์นี้ใช้ได้จริง · เหตุผลอยู่ที่ข้อมูล: เราสร้าง spread ด้วยกระบวนการ OU ที่มีแรงดึงกลับอยู่ในตัวโดยนิยาม (ดูบล็อกสร้างข้อมูลต้นบท) · เอากลยุทธ์ mean-reversion ไปวิ่งบนข้อมูลที่ mean-revert จริง แล้วได้กำไร — นั่นคือ *การตรวจว่าเครื่องมือวัดไม่เสีย* ไม่ใช่การค้นพบ

$$\frac{\text{Sharpe}}{\text{Calmar}} = \frac{R_{\text{ann}} - R_f}{\sigma_{\text{ann}}} \quad \text{Calmar} = \frac{R_{\text{ann}}}{\text{MaxDD}}$$

คำถามที่ต้องถามต่อจึงไม่ใช่ "Sharpe พอไหม" แต่คือ "ตลาดจริงมีแรงดึงกลับแบบที่เราใส่ลงไปหรือเปล่า และมันอยู่ได้นานแค่ไหน" — บทที่ 24 (Walk-Forward) มีไว้ตอบข้อนี้

● [รอบสอง] Sharpe 1.775 บนข้อมูลจำลอง แปลว่าอะไรกันแน่

ลำดับของเหตุผลตรงนี้สำคัญกว่าตัวเลขทุกตัวในบท · เราเป็นคนใส่ edge ลงไปในข้อมูลเอง ด้วยบรรทัด `ou[t] = ou[t-1] + kappa * (0 - ou[t-1]) + ... · kappa = 0.15` คือแรงดึงกลับที่เรากำหนด ถ้ากลยุทธ์ไม่ทำกำไรบนข้อมูลแบบนี้ แปลว่ามีบั๊กในโค้ด

ดังนั้นบทบาทที่ถูกต้องของตัวเลขชุดนี้คือ **การทดสอบเชิงบวก (positive control)** — เหมือนที่ห้องแล็บใส่ตัวอย่างที่รู้ผลอยู่แล้วเข้าเครื่องเพื่อดูว่าเครื่องอ่านค่าได้ · มันตอบว่า "backtest นี้จับ edge ที่มีอยู่จริงได้" ไม่ได้ตอบว่า "มี edge ในตลาดจริง"

❗ **คู่ที่ควรทำเสมอ:** รัน backtest ตัวเดียวกันบนข้อมูลที่ไม่มี edge (เช่น random walk ล้วน ไม่มี OU) ด้วย · ถ้ามันยังกำไรอยู่ แปลว่าโค้ดรั่วที่ไหนสักแห่ง · การมีทั้ง positive control และ negative control คือความต่างระหว่าง "รันแล้วได้เลข" กับ "รู้ว่าเลขนั้นเชื่อถือ"

⚠ กับดักที่ตามมาทันที: พอเห็น Sharpe 1.775 สัญชาตญาณคือ "ลองปรับ window กับ entry_z ให้ดีกว่านี้อีก" — ระวังให้ดี · การไล่จนพารา มิเตอร์บนชุดข้อมูลเดียวกันจนตัวเลขสวยขึ้นเรื่อย ๆ คือ **overfit ที่เนบเนียนที่สุด** และบนข้อมูลจำลองที่เรารู้คำตอบอยู่แล้ว มันยังไร้ความหมาย · บทที่ 24 มีไว้บอกว่าค่าที่จูนมานั้นจริงหรือแค่ฟลุค

22.4 Equity Curve และ Drawdown

```
def summarize_equity(result: pd.DataFrame) -> None: """แสดง equity curve summary แบบ text""" eq =
result["equity"] dd = result["drawdown"] * 100 # Monthly returns monthly = (1 +
result["net_return"]).resample("ME").prod() - 1 print("Monthly Returns (%):") print(" " + " ".join( f"
{d.strftime('%b%y')}: {r*100:+5.1f}%" for d, r in monthly.items() )[:120]) print(f"\nEquity: start=
{eq.iloc[0]:.4f} " f"peak={eq.max():.4f} " f"end={eq.iloc[-1]:.4f}") print(f"Drawdown: worst={dd.min():.2f}% "
f"avg_when_down={dd[dd<0].mean():.2f}%") # Time spent in drawdown pct_in_dd = (dd < -1).mean() * 100
print(f"Time in drawdown (>1%): {pct_in_dd:.1f}% of trading days") summarize_equity(result)
```

► OUTPUT

Monthly Returns (%):

```
Mar23: +2.2% Apr23: +1.7% May23: +0.6% Jun23: +1.1% Jul23: +0.0% Aug23: +1.2% Sep23: +0.7% Oct23: +0.8% Nov23: +
Equity: start=1.0000 peak=1.1348 end=1.1325
Drawdown: worst=-2.24% avg_when_down=-0.26%
Time in drawdown (>1%): 1.4% of trading days
```

ตัวอย่าง: Parameter Sweep แบบ Vectorized

ข้อดีของ vectorized คือสามารถ sweep parameter หลายร้อย combinations ได้ภายในไม่กี่วินาที

```
def parameter_sweep(price_a, price_b, windows, entry_zs, fee_rate=0.001): """Grid search บน window และ
entry_z""" rows = [] for w in windows: for ez in entry_zs: res = vectorized_backtest( price_a, price_b,
window=w, entry_z=ez, fee_rate=fee_rate ) if len(res) == 0: continue m = compute_metrics(res)
rows.append({ "window": w, "entry_z": ez, "sharpe": m["Sharpe Ratio"], "calmar": m["Calmar Ratio"],
"max_dd": m["Max Drawdown (%)"], "n_active": int((res["signal"] != 0).sum()), }) df_out =
pd.DataFrame(rows) return df_out.sort_values("sharpe", ascending=False).reset_index(drop=True) sweep =
parameter_sweep( pd.Series(bybit, index=dates), pd.Series(binance, index=dates), windows = [30, 45, 60,
90, 120], entry_zs = [1.5, 2.0, 2.5, 3.0], ) print(sweep.head(8).to_string(index=False))
```

► OUTPUT

window	entry_z	sharpe	calmar	max_dd	n_active
120	2.0	2.276	7.497	-1.07	71
120	1.5	2.075	4.401	-2.05	112
90	2.0	2.074	3.887	-2.20	94
45	2.5	1.947	3.765	-1.44	51
90	2.5	1.884	6.043	-0.75	23
60	1.5	1.811	3.724	-2.24	144
60	2.0	1.775	3.296	-2.24	107
60	2.5	1.593	2.208	-2.20	35

► แบบฝึกหัด: เพิ่ม Sortino Ratio ใน compute_metrics

บทที่ 23: Event-Driven Backtest

แก่นของบทนี้

Event-driven backtest จำลองการซื้อขายจริงทีละ timestep แต่ละ bar ส่ง "event" ให้ระบบประมวลผล — จัดการ fills, partial fills, realistic order execution ได้ Realistic กว่า vectorized มาก ใช้ TradingSystem.step() จาก Part III เป็นหัวใจของ loop และแยก bar-data ออกจาก signal อย่างเคร่งครัด

23.1 ความต่างระหว่าง Vectorized และ Event-Driven

คุณสมบัติ	Vectorized	Event-Driven
ความเร็ว	เร็วมาก (numpy)	ช้ากว่า 10–100x
ความสมจริง	ต่ำ — ไม่มี partial fill	สูง — จำลองได้ละเอียด
Lookahead bias	ต้องระวัง shift(1) เอง	ป้องกันโดยโครงสร้าง loop
Order types	Market order เท่านั้น	Limit, Stop, IOC ได้
Portfolio state	ยุ่งยากถ้า multi-asset	จัดการง่ายกว่า
ใช้เมื่อ	Parameter sweep เบื้องต้น	Final validation ก่อน live

23.2 Data Structures สำหรับ Event System

📖อ่านว่า: บล็อกข้างล่างไม่มีตรรกะให้คิดตามเลยสักบรรทัด — มันคือการประกาศว่า "ในระบบนี้มีของอยู่ 5 ชนิด และแต่ละชนิดมีช่องอะไรบ้าง" · อ่านเหมือนอ่านสารบัญ: BarData = ราคา 1 แท่ง · Signal = สิ่งที่ถูกกลยุทธ์อยากทำ · Fill = สิ่งที่เกิดขึ้นจริงหลังส่งคำสั่ง · PortfolioState = สภาพพอร์ต ณ ขณะนั้น · ความต่างระหว่าง Signal กับ Fill คือหัวใจของทั้งบท — สิ่งที่ยากทำกับสิ่งที่ได้จริงไม่เท่ากัน และช่องว่างตรงนั้นคือที่ที่เงินหายไป

```
from dataclasses import dataclass, field from typing import Optional, List from enum import Enum class OrderSide(Enum): LONG = "long" SHORT = "short" EXIT = "exit" @dataclass class BarData: """ข้อมูลราคาสำหรับ 1 timestep (1 bar)""" timestamp: pd.Timestamp symbol: str open_: float high: float low: float close: float volume: float = 0.0 @dataclass class Signal: """Output จาก strategy.step()""" timestamp: pd.Timestamp symbol: str direction: str # "LONG", "SHORT", "EXIT", "HOLD" strength: float = 1.0 @dataclass class Fill: """ผลการ execute order จริง""" timestamp: pd.Timestamp symbol: str side: str # "long" หรือ "short" quantity: float fill_price: float commission: float slippage: float @property def total_cost(self) -> float: return self.commission + self.slippage @dataclass class PortfolioState: """State ของ portfolio ณ เวลานั้น""" cash: float positions: dict = field(default_factory=dict) # symbol -> qty avg_prices: dict = field(default_factory=dict) # symbol -> avg cost realized_pnl: float = 0.0 unrealized_pnl: float = 0.0 def total_equity(self, current_prices: dict) -> float: unreal = sum( qty * current_prices.get(sym, self.avg_prices.get(sym, 0)) for sym, qty in self.positions.items() ) return self.cash + unreal
```

```
@dataclass เขียนโค้ดให้เราฟรี ๆ ก็บรรทัด @dataclass class Signal: timestamp: pd.Timestamp symbol: str direction: str strength: float = 1.0 # ↑ เท่ากับเขียนเองแบบนี้ทั้งหมด:
```

```
# def __init__(self, timestamp, symbol, direction, strength=1.0): ...
# def __repr__(self): ...      - พิมพ์ออกมาแล้วอ่านรู้เรื่อง
# def __eq__(self, other): ... - เทียบ == แล้วดูค่าข้างใน ไม่ใช่ที่อยู่หน่วยความจำ
```

1. สิ่งที่ได้ฟรีที่มีค่าที่สุดคือ `__eq__` — object ธรรมดาสองตัวที่มีค่าเหมือนกันเป๊ะจะยังถือว่า *ไม่เท่ากัน* เพราะ Python เทียบที่อยู่หน่วยความจำ · พอเป็น dataclass มันเทียบค่าข้างในทีละช่อง ซึ่งทำให้เขียน test แบบ `assert got == Signal(...)` ได้ตรงๆ
2. `open_`: float มีขีดล่างต่อท้ายเพราะ `open` เป็นชื่อฟังก์ชันของ Python เอง · ตั้งชื่อทับได้จริงและจะไม่ error ทันที แต่จะไปพึ่งตอนใครสักคนเรียก `open()` เพื่อเปิดไฟล์ในไฟล์เดียวกัน · ขีดล่างต่อท้ายคือธรรมเนียมมาตรฐานสำหรับกรณีนี้ (เจอบ่อยกับ `class_`, `id_`, `type_`)
3. `field(default_factory=dict)` — จุดนี้สำคัญมากอ่านให้ดี · เขียน `positions: dict = {}` ตรงๆไม่ได้ เพราะค่าเริ่มต้นใน Python ถูกสร้างครั้งเดียวตอนนิยามคลาส ไม่ใช่ทุกครั้งที่สร้าง object · ผลคือทุก `PortfolioState` ที่สร้างขึ้นจะใช้ `dict` ก้อนเดียวกันร่วมกันทั้งหมด — พอร์ตหนึ่งเปิดสถานะ อีกพอร์ตก็เห็นด้วย · `default_factory=dict` แปลว่า "เรียก `dict()` ใหม่ทุกครั้งที่เราสร้าง" · dataclass ถึงกับโยน error ทันทีถ้าคุณใส่ `dict/list` เป็นค่าเริ่มต้นตรงๆ เพราะกับดักนี้ทำคนตกม้าเยอะเกินไป
4. `@property total_cost` ใน `Fill` — dataclass ก็ยังใส่ property เองได้ตามปกติ · `commission + slippage` คำนวณสดทุกครั้ง จึงไม่มีวันไม่ตรงกับสองช่องนั้น (แนวคิดเดียวกับ `Position.pnl` ในบทที่ 18)

● เทียบกับ Portfolio ในบทที่ 21 — คนละวิธีเก็บ "ทิศทาง"

สังเกต `PortfolioState.positions` ว่าเป็น `symbol -> qty` เฉยๆ ไม่มีช่อง `side` เลย · ทิศทางถูกเก็บไว้ในเครื่องหมายของจำนวน — long คือบวก short คือลบ

ผลที่ตามมาคือ `total_equity` เขียนได้สั้นๆ ว่า `qty * price` แล้วถูกต้องทั้งสองฝั่งโดยอัตโนมัติ ไม่ต้องมี `if side == "long"` ที่ไหนเลย · เทียบกับ `Portfolio` ในบทที่ 21 ที่เก็บ `side` เป็นข้อความแยกต่างหาก แล้วต้องคูณ `sign` เองในทุกเมธอดที่ขยับเงิน — สิมที่เดียวก็คิดบัญชี short กลับทิศทันที

✖ **บทเรียนที่ใหญ่กว่าเรื่อง short:** ถ้าข้อมูลชิ้นหนึ่งต้องถูกตีความเหมือนกันทุกที่ที่ใช้ให้ออกแบบให้การตีความนั้นฝังอยู่ในตัวข้อมูลเอง อย่าปล่อยให้มันเป็นหน้าที่ของคนเขียนโค้ดที่ต้องจำให้ครบทุกจุด · ตัวเลขติดลบจำแทนเราได้ แต่ `if` ที่กระจายอยู่ 5 ที่ต้องอาศัยความจำของคน

23.3 Event-Driven Loop ที่ใช้ `TradingSystem.step()`

🧱 โครงก่อนโค้ด — 170 บรรทัด แต่มีวิธีอ่านที่ทำให้เหลือ 4 ก่อน

```
class EventDrivenBacktester:
    def __init__(...)          # ตั้งค่า + จองที่เก็บสถานะ

    def _compute_zscore(...)   1
    def _compute_zscore_from_prices(...) | ผู้ช่วย — เรียกจาก run() เท่านั้น
    def _generate_signal(...)   | (ขีดล่างหน้าหน้า = ของภายใน)
    def _execute_signal(...)    1

    def run(df_a, df_b)        # - เริ่มอ่านที่นี่ ไม่ใช่บรรทัดแรก
        for แต่ละแท่ง:
            ① รับ MarketEvent      # เห็นราคาใหม่ 1 แท่ง
            ② อัปเดต signal         # คัด z-score -> ตัดสินใจ
            ③ Execute signal        # แปลงการตัดสินใจเป็น Fill
            ④ Mark-to-market equity # ตีมูลค่าพอร์ต ณ ราคาปิด
```

1. อ่านจาก `run()` ก่อนเสมอ — อย่าไล่จากบรรทัดแรก · `run()` คือโครงเรื่อง ส่วนเมธอด — ทั้งสี่คือรายละเอียดที่มันเรียกใช้ · ถ้าไล่จากบนลงล่าง คุณจะเจอรายละเอียดก่อนที่จั่วว่ามันถูกใช้ทำอะไร ซึ่งเป็นวิธีที่ทำให้หลงเร็วที่สุด
2. ทั้งบล็อกไม่มีแนวคิดใหม่เลยแม้แต่ตัวเดียว — `z-score` (บทที่ 12) · `signal` จาก `entry/exit` (บทที่ 22) · dataclass (หัวข้อที่แล้ว) · คลาสและ `self` (Part III) · งานของบล็อกนี้คือ "ต่อสายให้ถูก" ไม่ใช่สอนของใหม่ ถ้าตรงไหนอ่านไม่ออก ให้กลับไปบทเจ้าของเรื่อง ไม่ใช่ผินอ่านต่อ
3. ความต่างจาก `vectorized` ทั้งหมดอยู่ที่ `for` ตัวเดียว · แบบ `vectorized` คิดทั้งคอลัมน์รวดเดียว จึงต้องระวัง `shift(1)` เอง · แบบนี้เดินทีละแท่งและ ณ แท่งที่ `t` ใ้คมีข้อมูลถึง `t` เท่านั้นจริง ๆ — การมองอนาคตจึงถูกกันโดยโครงสร้าง ไม่ใช่โดยวินัยของคนเขียน · แลกมาด้วยความเร็วที่ช้ากว่าหลายสิบเท่า

ลำดับ ①→④ ในลูปห้ามสลับ. โดยเฉพาะ ④ ต้องอยู่หลัง ③ – ตีมูลค่าพอร์ตก่อนที่จะคำสั่งของแท่งนั้นจะถูก execute เท่ากับรายงาน equity ของสถานะที่ยังไม่มีจริง

```
class EventDrivenBacktester: """ Event-driven backtester ที่ใช้ TradingSystem จาก Part III ทุก bar → MarketEvent → Strategy.step() → Signal → Execution → Fill """ def __init__(self, window: int = 60, entry_z: float = 2.0, exit_z: float = 0.5, fee_rate: float = 0.001, slippage_bps: float = 5.0, initial_capital: float = 1_000_000.0): self.window = window self.entry_z = entry_z self.exit_z = exit_z self.fee_rate = fee_rate self.slippage_bps = slippage_bps # basis points self.initial_capital = initial_capital # State self.cash = initial_capital self.position = 0.0 # +1 = long spread, -1 = short spread, 0 = flat self.entry_price = None self.bar_history: List[BarData] = [] # Records self.fills: List[Fill] = [] self.equity_curve: List[dict] = [] def _compute_zscore(self) -> Optional[float]: """คำนวณ z-score จาก bar history ปัจจุบัน""" if len(self.bar_history) < self.window: return None recent = self.bar_history[-self.window:] spreads = [np.log(b.close / b.volume) if b.volume > 0 else 0.0 for b in recent] # (volume ที่นี้ใช้แทน price_b เพื่อความง่าย) arr = np.array(spreads) std = arr.std() return float((arr[-1] - arr.mean()) / std) if std > 0 else 0.0 def _compute_zscore_from_prices(self, closes_a: np.ndarray, closes_b: np.ndarray) -> float: """คำนวณ z-score จาก price arrays สองตัว""" if len(closes_a) < self.window: return 0.0 window_a = closes_a[-self.window:] window_b = closes_b[-self.window:] spread = np.log(window_a) - np.log(window_b) std = spread.std() return float((spread[-1] - spread.mean()) / std) if std > 0 else 0.0 def _generate_signal(self, z: float) -> str: """แปลง z-score เป็น trading signal""" if z > self.entry_z: return "SHORT" # spread กว้าง → short if z < -self.entry_z: return "LONG" # spread แคบ → long if abs(z) < self.exit_z and self.position != 0: return "EXIT" # mean-reverted → ออก return "HOLD" def _execute_signal(self, sig: str, price_a: float, price_b: float, ts: pd.Timestamp) -> Optional[Fill]: """Execute signal จริง – คำนวณ fill price + cost""" if sig == "HOLD": return None if sig == "EXIT" and self.position == 0: return None if sig in ("LONG", "SHORT") and self.position != 0: return None # มี position อยู่แล้ว # Slippage: ราคา fill แยกว่า close เล็กน้อย slip_mult = self.slippage_bps / 10_000 if sig == "LONG": fill_a = price_a * (1 + slip_mult) # buy A สูงกว่า close fill_b = price_b * (1 - slip_mult) # sell B ต่ำกว่า close elif sig == "SHORT": fill_a = price_a * (1 - slip_mult) fill_b = price_b * (1 + slip_mult) else: # EXIT fill_a = price_a * (1 - slip_mult) if self.position > 0 else 1 + slip_mult fill_b = price_b * (1 + slip_mult) if self.position > 0 else 1 - slip_mult # ขนาด position (10% ของ cash) notional = self.cash * 0.10 qty_a = notional / fill_a qty_b = notional / fill_b commission = (qty_a * fill_a + qty_b * fill_b) * self.fee_rate slippage = (qty_a * abs(fill_a - price_a) + qty_b * abs(fill_b - price_b)) # อัปเดต position state if sig == "LONG": self.position = 1.0 self.entry_price = (fill_a, fill_b, qty_a, qty_b) elif sig == "SHORT": self.position = -1.0 self.entry_price = (fill_a, fill_b, qty_a, qty_b) else: # EXIT if self.entry_price is not None: ep_a, ep_b, qy_a, qy_b = self.entry_price if self.position == 1.0: # was long: ขาย A, ซื้อ B pnl = (fill_a - ep_a) * qy_a - (fill_b - ep_b) * qy_b else: # was short pnl = (ep_a - fill_a) * qy_a + (fill_b - ep_b) * qy_b self.cash += pnl - commission self.position = 0.0 self.entry_price = None return Fill( timestamp = ts, symbol = "BTC-Pair", side = sig.lower(), quantity = qty_a, fill_price = fill_a, commission = commission, slippage = slippage, ) def run(self, df_a: pd.Series, df_b: pd.Series) -> pd.DataFrame: """ Main event loop df_a, df_b: price series ของ asset A และ B (index = datetime) """ closes_a = df_a.values closes_b = df_b.values timestamps = df_a.index n = len(closes_a) for i in range(n): ts = timestamps[i] price_a = float(closes_a[i]) price_b = float(closes_b[i]) # — ขั้นตอนที่ 1: รับ MarketEvent ————— # (ใน live system: รอ data จาก exchange feed) # — ขั้นตอนที่ 2: อัปเดต signal ————— # ใช้ข้อมูลถึง i (ไม่รวม i+1) → ไม่มี lookahead bias if i < self.window: self.equity_curve.append({"ts": ts, "equity": self.cash}) continue z = self._compute_zscore_from_prices(closes_a[i:], closes_b[i:]) sig = self._generate_signal(z) # — ขั้นตอนที่ 3: Execute signal ————— fill = self._execute_signal(sig, price_a, price_b, ts) if fill is not None: self.fills.append(fill) # — ขั้นตอนที่ 4: Mark-to-market equity ————— unreal = 0.0 if self.position != 0 and self.entry_price is not None: ep_a, ep_b, qy_a, qy_b = self.entry_price if self.position == 1.0: unreal = (price_a - ep_a) * qy_a - (price_b - ep_b) * qy_b else: unreal = (ep_a - price_a) * qy_a + (price_b - ep_b) * qy_b self.equity_curve.append({"ts": ts, "equity": self.cash + unreal, "signal": sig, "zscore": z, "position": self.position, }) return pd.DataFrame(self.equity_curve).set_index("ts") # — รับ Event-Driven Backtest ————— bt_ed = EventDrivenBacktester( window=60, entry_z=2.0, exit_z=0.5, fee_rate=0.001, slippage_bps=5.0, initial_capital=1_000_000, ) ed_result = bt_ed.run( df_a = pd.Series(bybit, index=dates), df_b = pd.Series(binance, index=dates), ) print(f"Fills executed: {len(bt_ed.fills)}") print(f"Final equity: {bt_ed.cash:,.0f}") print(f"Return: {bt_ed.cash / bt_ed.initial_capital - 1:+.2%}") print(f"\nLast 3 fills:") for f in bt_ed.fills[-3:]: print(f"{f.timestamp.date()} {f.side:5s} " f"qty={f.quantity:.4f} @ {f.fill_price:,.0f} " f"comm={f.commission:.2f} slip={f.slippage:.2f}")
```

► OUTPUT

```
Fills executed: 28
Final equity: 1,014,251
Return: +1.43%
```

Last 3 fills:

```
2024-03-22 exit qty=2.7597 @ 36,687 comm=202.49 slip=101.25
2024-04-24 short qty=2.7385 @ 36,975 comm=202.52 slip=101.26
2024-05-03 exit qty=2.8300 @ 35,780 comm=202.52 slip=101.26
```

23.4 ทำไม Event-Driven ให้ผลต่างจาก Vectorized

```
# เปรียบเทียบ vectorized vs event-driven บนข้อมูลเดียวกัน price_a_s = pd.Series(bybit, index=dates) price_b_s =
pd.Series(binance, index=dates) # Vectorized vec_result = vectorized_backtest(price_a_s, price_b_s, window=60,
entry_z=2.0, fee_rate=0.001) vec_metrics = compute_metrics(vec_result) # Event-driven ed_return = bt_ed.cash /
bt_ed.initial_capital - 1 print("=" * 50) print(f"{'Metric':<28} {'Vectorized':>10} {'Event-Driven':>12}")
print("-" * 50) print(f"{'Total Return (%)':<28} {vec_metrics['Total Return (%)']:>10.2f} " f"
{ed_return*100:>12.2f}") print(f"{'Sharpe Ratio':<28} {vec_metrics['Sharpe Ratio']:>10.3f} " f"{'(see
below)':>12}") print(f"{'Num fills':<28} {int((vec_result['signal'].diff().abs())>0).sum():>10} "
f"{len(bt_ed.fills):>12}")
```

► OUTPUT

```
=====
Metric                                Vectorized  Event-Driven
-----
Total Return (%)                      13.25         1.43
Sharpe Ratio                          1.775      (see below)
Num fills                             28            28
```

ทำไม Event-Driven ให้ผลต่ำกว่า?

- **Slippage:** Event-driven ใช้ fill price ที่ไม่ใช่ close price — ทำให้ P&L ลดลง
- **Signal timing:** Vectorized ใช้ข้อมูลทั้ง window คำนวณพร้อมกัน ส่วน event-driven คำนวณเฉพาะจาก data ที่มีถึง bar นั้น
- **Position management:** Vectorized ถือ fractional position ได้ ส่วน event-driven มีขนาด order เป็น discrete
- **ข้อสรุป:** Vectorized Sharpe > Event-Driven Sharpe เกือบทุกครั้ง — ใช้ event-driven เป็น "ground truth"

► แบบฝึกหัด: เพิ่ม stop-loss ใน EventDrivenBacktester

บทที่ 24: Walk-Forward Validation

แก่นของบทนี้

Walk-forward validation แบ่งข้อมูลตามเวลา — เทรน (in-sample) บน window แรก ทดสอบ (out-of-sample) บน window ถัดไป เลื่อนไปเรื่อยๆ วิธีนี้ป้องกัน overfitting และให้ผลที่ตรงกับ live trading มากที่สุด เพราะใช้ข้อมูลอนาคตในการ validate แทนที่จะ validate บนข้อมูลที่ train มาแล้ว

24.1 Overfitting Trap — Sharpe 2.0 กลายเป็นติดลบ

Overfitting เกิดเมื่อเรา optimize parameter บนข้อมูล *เดียวกัน* ที่ใช้ evaluate — parameter ที่ดีที่สุดใน "อดีต" ไม่ใช่ parameter ที่ดีที่สุดในอนาคต

```
# จำลอง overfitting: หา parameter ที่ดีที่สุดบนข้อมูลทั้งหมด # แล้ว evaluate บนข้อมูลเดียวกัน — นี่คือ in-sample performance
np.random.seed(0) n_total = 1000 dates_long = pd.date_range("2021-01-01", periods=n_total, freq="D") # สร้างข้อมูลที่
pair trading edge อ่อนมาก (จงใจ) btc_base = 30000 * np.exp(np.cumsum(np.random.normal(0.001, 0.03, n_total))) #
⚠️ ตรงนี้ "จงใจ" ให้ spread เป็น noise ล้วน — ไม่มี mean-reversion จริง # ต่างจากบทที่ 22 ที่ใช้ OU เพราะที่นั่นต้องการ edge จริงเพื่อ
โซว์ backtest # แต่ตอนนี้ต้องการ edge "ปลอม" เพื่อให้เห็นว่า overfitting หน้าตาเป็นอย่างไร # → ถ้าข้อมูลมี edge จริง พารามิเตอร์ที่ดีในอดีต
ก็จะดีในอนาคตด้วย = ไม่ decay noise_scale = 120.0 bybit_l = btc_base + np.random.normal(0, noise_scale, n_total)
binance_l = btc_base + np.random.normal(0, noise_scale, n_total) pa = pd.Series(bybit_l, index=dates_long) pb =
pd.Series(binance_l, index=dates_long) # — แบ่งข้อมูลก่อนทำอะไรทั้งสิ้น — # ⚠️ จุดตายของการทดสอบแบบนี้: ถ้า optimize บน
ข้อมูล "ทั้งหมด" แล้วเอา # ท่อนท้ายมาเรียกว่า out-of-sample มันไม่ใช่ out-of-sample เลย # เพราะท่อนนั้นถูกใช้เลือก parameter ไปแล้ว →
decay ที่วัดได้ไร้ความหมาย train_a, train_b = pa.iloc[:800], pb.iloc[:800] test_a, test_b = pa.iloc[800:],
pb.iloc[800:] # In-sample: optimize บน 800 วันแรกเท่านั้น in_sample_sweep = parameter_sweep(train_a, train_b,
windows = [30, 60, 90, 120], entry_zs = [1.5, 2.0, 2.5, 3.0]) best_params = in_sample_sweep.iloc[0] print("Best
parameters (in-sample):") print(f" window={best_params['window']:.0f}, " f"entry_z=
{best_params['entry_z']:.1f}") print(f" In-sample Sharpe: {best_params['sharpe']:.2f}") # Out-of-sample:
parameter เดิม แต่ test บน 200 วันสุดท้ายที่ยังไม่เคยเห็น oos_res = vectorized_backtest( test_a, test_b, window =
int(best_params["window"]), entry_z = best_params["entry_z"], ) oos_m = compute_metrics(oos_res) print(f" Out-
of-sample Sharpe: {oos_m['Sharpe Ratio']:.2f}") kept = oos_m["Sharpe Ratio"] / best_params["sharpe"] print(f"
เหลือจาก in-sample: {kept:.1%}" f"{' - ติดลบ = พลิกเป็นขาดทุน' if kept < 0 else ''}")
```

► OUTPUT

```
Best parameters (in-sample):
  window=120, entry_z=1.5
In-sample Sharpe: 4.77
Out-of-sample Sharpe: 2.20
เหลือจาก in-sample: 46.2%
```

Overfitting เกิดจากอะไร?

เราทดสอบ 16 parameter combinations บนข้อมูลที่ **ไม่มี edge จริงเลย** — แต่ในบรรดา 16 ชุดนั้นย่อมมีชุดที่ "ดูดี" โดยบังเอิญ · การหิบบชุดที่ดีที่สุดมาใช้ = การหิบบ **ความบังเอิญที่แรงที่สุด** ไม่ใช่ edge · พอ data เปลี่ยน ความบังเอิญชุดนั้นก็หายไป เหลือแต่ความบังเอิญชุดใหม่ ซึ่งอาจสูงหรือต่ำก็ได้ — ที่แน่นอนคือมัน **ไม่เกี่ยวกับพารามิเตอร์ที่เราทดสอบ** เลือกมา

ยิ่งลองพารามิเตอร์มาก โอกาสเจอ "ความบังเอิญที่ดูดี" ยิ่งสูง — นี่คือเหตุผลที่จำนวนชุดที่ทดสอบต้องถูกนับเป็นต้นทุน (multiple testing) ไม่ใช่ของฟรี

📌 **อ่านว่า:** Sharpe 4.77 ในอดีต → 2.20 ในอนาคต · เหลือ 46% · ดูเหมือน "เสื่อมลงแต่ยังใช้ได้" — แต่ตัวเลขทั้งสองตัว **ไม่มีความหมายเลย** เพราะข้อมูลชุดนี้ถูกสร้างมาให้ **ไม่มี edge** (spread เป็น noise ล้วน ดูคอมเมนต์ในบล็อกสร้างข้อมูล) · Sharpe 4.77 คือความบังเอิญที่แรงที่สุดในบรรดา 16 ชุดที่เราลอง และ 2.20 คือความบังเอิญของช่วง 200 วันสุดท้ายที่มีเทรดอยู่ไม่กี่ครั้ง

นี่คือหน้าตาที่แท้จริงของ overfitting — ไม่ได้พังโครมคราม แต่ให้ตัวเลขที่ยัง "พอรับได้" จนคุณกล้าเอาไปใช้จริง

⚠ อีกกับดักในตาราง walk-forward ข้างล่าง — `oos_sharpe = 0.000` ไม่ได้แปลว่า "เท่าทุน"

หลาย fold จะได้ `oos_sharpe = 0.000` พร้อม `oos_return = 0.00` — นั่นแปลว่า **ไม่มีเทรดเลยในช่วงนั้น** ไม่ใช่ "เทรดแล้วเสมอตัว" . ถ้าเอาค่า 0 พวกนี้ไปหาค่าเฉลี่ยรวมกับ fold อื่น คุณจะได้อัตราผลตอบแทนที่ดูดีแต่จริงๆ แล้วมันไม่มีค่าอะไรเลย

🔍 **นิสัยที่ต้องติดตัว:** ดูจำนวนเทรดคู่กับ Sharpe เสมอ — Sharpe จาก 5 เทรดกับจาก 200 เทรดไม่ควรถูกอ่านด้วยสายตาเดียวกัน . ถ้า out-of-sample มีเทรดน้อยกว่า ~30 ครั้ง ให้ถือว่าตัวเลขนั้นเป็นสัญญาณรบกวนจนกว่าจะพิสูจน์เป็นอย่างอื่น

24.2 Walk-Forward ด้วย Expanding Window

Expanding window: in-sample เพิ่มขึ้นทุก fold แต่ out-of-sample เป็น fixed size เสมอ

Walk-Forward — Expanding Window

Fold 1: [TRAIN | TEST] Fold 2: [TRAIN | TEST] Fold 3: [TRAIN | TEST]

expanding fixed Walking Window (Rolling): Fold 1: [TRAIN | TEST] Fold 2: [TRAIN | TEST] Fold 3: [TRAIN | TEST] ← fixed window size → ← fixed →

```
from sklearn.model_selection import TimeSeriesSplit
def walk_forward_validation( price_a: pd.Series, price_b: pd.Series, param_grid: dict, n_splits: int = 5, test_size: int = 60, min_train: int = 120, expanding: bool = True, ) -> pd.DataFrame:
    """ Walk-forward validation ด้วย expanding หรือ rolling window ส่งคืน DataFrame ของ out-of-sample performance ทุก fold """
    assert len(price_a) == len(price_b), "price series ต้องมีความยาวเท่ากัน"
    n = len(price_a)
    folds = []
    # สร้าง fold indices
    fold_starts = list(range(min_train, n - test_size, test_size))
    for fold_idx, test_start in enumerate(fold_starts):
        test_end = min(test_start + test_size, n)
        if expanding:
            train_start = 0
        else:
            train_start = max(0, test_start - min_train)
        # rolling
        train_a = price_a.iloc[train_start:test_start]
        train_b = price_b.iloc[train_start:test_start]
        test_a = price_a.iloc[test_start:test_end]
        test_b = price_b.iloc[test_start:test_end]
        # — In-sample: หา best parameters
        best_sharpe = -np.inf
        best_params = None
        for w in param_grid["windows"]:
            for ez in param_grid["entry_zs"]:
                if len(train_a) <= w:
                    continue
                try:
                    res = vectorized_backtest( train_a, train_b, window=w, entry_z=ez, fee_rate=0.001 )
                    if len(res) == 0:
                        continue
                    m = compute_metrics(res)
                    if m["Sharpe Ratio"] > best_sharpe:
                        best_sharpe = m["Sharpe Ratio"]
                        best_params = {"window": w, "entry_z": ez}
                except Exception:
                    continue
        # — Out-of-sample: test ด้วย best parameters
        try:
            oos_res = vectorized_backtest( test_a, test_b, window = best_params["window"], entry_z = best_params["entry_z"], fee_rate = 0.001, )
            oos_m = compute_metrics(oos_res)
            if len(oos_res) > 0 else {}
        except Exception:
            oos_m = {}
        folds.append({ "fold": fold_idx + 1, "train_start": price_a.index[train_start].date(), "train_end": price_a.index[test_start - 1].date(), "test_start": price_a.index[test_start].date(), "test_end": price_a.index[test_end - 1].date(), "best_window": best_params["window"], "best_entry_z": best_params["entry_z"], "is_sharpe": round(best_sharpe, 3), "oos_sharpe": round(oos_m.get("Sharpe Ratio", 0), 3), "oos_return": round(oos_m.get("Total Return (%)", 0), 2), "oos_max_dd": round(oos_m.get("Max Drawdown (%)", 0), 2), })
    return pd.DataFrame(folds)
# — รัน Walk-Forward
wf_result = walk_forward_validation( price_a = pa, price_b = pb, param_grid = { "windows": [30, 60, 90, 120], "entry_zs": [1.5, 2.0, 2.5, 3.0], }, n_splits = 5, test_size = 100, min_train = 200, expanding = True, )
print(wf_result.to_string(index=False))
```

► OUTPUT

fold	train_start	train_end	test_start	test_end	best_window	best_entry_z	is_sharpe	oos_sharpe	oos_return	oos_max_dd
1	2021-01-01	2021-07-19	2021-07-20	2021-10-27	30	1.5	3.288	1.854	3.19	-1.6
2	2021-01-01	2021-10-27	2021-10-28	2022-02-04	120	1.5	3.401	0.000	0.00	0.6
3	2021-01-01	2022-02-04	2022-02-05	2022-05-15	120	1.5	2.965	0.000	0.00	0.6
4	2021-01-01	2022-05-15	2022-05-16	2022-08-23	120	1.5	3.047	0.000	0.00	0.6
5	2021-01-01	2022-08-23	2022-08-24	2022-12-01	120	1.5	3.133	0.000	0.00	0.6
6	2021-01-01	2022-12-01	2022-12-02	2023-03-11	120	1.5	4.092	0.000	0.00	0.6
7	2021-01-01	2023-03-11	2023-03-12	2023-06-19	120	1.5	4.767	0.000	0.00	0.6

24.3 วิเคราะห์ผล Walk-Forward

```
def analyze_walk_forward(wf: pd.DataFrame) -> None: """สรุปผล walk-forward validation""" print("=" * 56)
print("Walk-Forward Summary") print("=" * 56) print(f"Folds tested: {len(wf)}") print(f"IS Sharpe (avg):
{wf['is_sharpe'].mean():.3f} " f"± {wf['is_sharpe'].std():.3f}") print(f"OOS Sharpe (avg):
{wf['oos_sharpe'].mean():.3f} " f"± {wf['oos_sharpe'].std():.3f}") ratio = wf['oos_sharpe'].mean() /
wf['is_sharpe'].mean() print(f"OOS/IS Ratio: {ratio:.1%} " f"({'acceptable' if ratio > 0.5 else 'likely
overfit'})") print(f"OOS Sharpe > 0: " f"{(wf['oos_sharpe'] > 0).sum()}/{len(wf)} folds") print(f"OOS Sharpe >
1: " f"{(wf['oos_sharpe'] > 1.0).sum()}/{len(wf)} folds") print(f"OOS Return (avg):
{wf['oos_return'].mean():.2f}%") print(f"OOS Max DD (avg): {wf['oos_max_dd'].mean():.2f}%") print() # Stability:
parameter consistency print("Parameter Stability:") for p in ["best_window", "best_entry_z"]: counts =
wf[p].value_counts() most_common = counts.index[0] pct = counts.iloc[0] / len(wf) * 100 print(f" {p}: most
common = {most_common} ({pct:.0f}% of folds)") analyze_walk_forward(wf_result)
```

► OUTPUT

Walk-Forward Summary

```
Folds tested:          7
IS Sharpe (avg):       3.528 ± 0.662
OOS Sharpe (avg):       0.265 ± 0.701
OOS/IS Ratio:          7.5% (likely overfit)
OOS Sharpe > 0:        1/7 folds
OOS Sharpe > 1:        1/7 folds
OOS Return (avg):       0.46%
OOS Max DD (avg):       -0.15%
Parameter Stability:
  best_window: most common = 120 (86% of folds)
  best_entry_z: most common = 1.5 (100% of folds)
```

ตีความ OOS/IS RATIO

OOS/IS Ratio = 19.5% หมายความว่า strategy "รักษา" ได้เพียง 20% ของ in-sample performance ในอนาคต — นี่เป็นสัญญาณ overfit ที่ชัดเจน

$$\text{OOS/IS Ratio} = \frac{\text{OOS Sharpe (avg)}}{\text{IS Sharpe (avg)}}$$

- > 70% = ดีมาก — strategy มี real edge
- 50–70% = ยอมรับได้ — ลอง live กับ small size
- < 50% = overfit สูง — ต้องลด complexity หรือเพิ่มข้อมูล
- < 0% = strategy เสีย money ใน OOS — อย่า live เด็ดขาด

ตัวอย่าง: TimeSeriesSplit จาก scikit-learn

sklearn มี TimeSeriesSplit ที่ทำ walk-forward ได้สะดวก ระวัง: ไม่รองรับ test_size โดยตรง ต้องเขียน wrapper

```
from sklearn.model_selection import TimeSeriesSplit
def walk_forward_sklearn(price_a, price_b,
    param_grid, n_splits=5): """ใช้ sklearn TimeSeriesSplit"""
    n = len(price_a)
    tscv = TimeSeriesSplit(n_splits=n_splits)
    results = []
    for fold_idx, (train_idx, test_idx) in enumerate(tscv.split(price_a)):
        train_a = price_a.iloc[train_idx]
        train_b = price_b.iloc[train_idx]
        test_a = price_a.iloc[test_idx]
        test_b = price_b.iloc[test_idx]
        # หา best params ใน train
        best_sharpe = -np.inf
        best_p = None
        for w in param_grid["windows"]:
            for ez in param_grid["entry_zs"]:
                if len(train_a) <= w:
                    continue
                res = vectorized_backtest(train_a, train_b, window=w, entry_z=ez)
                if len(res) == 0:
                    continue
                sh = compute_metrics(res)["Sharpe Ratio"]
                if sh > best_sharpe:
                    best_sharpe, best_p = sh, {"window": w, "entry_z": ez}
        # Test oos
        oos = vectorized_backtest(test_a, test_b, **best_p)
        oos_m = compute_metrics(oos)
        if len(oos) > 0:
            oos_sharpe = oos_m.get("Sharpe Ratio", 0)
        else:
            oos_sharpe = None
        results.append({
            "fold": fold_idx + 1,
            "is_sharpe": round(best_sharpe, 3),
            "oos_sharpe": round(oos_sharpe, 3),
            "window": best_p["window"],
            "entry_z": best_p["entry_z"],
        })
    return pd.DataFrame(results)

wf_sk = walk_forward_sklearn(
    pa, pb, param_grid={"windows": [30, 60, 90], "entry_zs": [1.5, 2.0, 2.5]}, n_splits=5,
)
print(wf_sk.to_string(index=False))
```

► แบบฝึกหัด: เปรียบเทียบ expanding vs rolling window

บทที่ 25: Slippage, Fees & Reality

แก่นของบทนี้

Strategy ที่ดูดีใน backtest บ่อยครั้ง "หาย" ไปเมื่อใส่ต้นทุนจริง: bid-ask spread, commission, market impact สูตรคร่าวๆ:
$$\text{cost per trade} = \frac{\text{spread}}{2} + \text{commission} + \text{market_impact}$$
 บทนี้จะแสดงให้เห็นว่า strategy ที่ดูดี (Sharpe 2.67 เมื่อไม่คิด cost) เหลือ Sharpe 1.83 ที่ cost ต่ำ · 1.26 ที่ cost ระดับกลาง และ 0.21 ที่ cost สูง — กลยุทธ์เดียวกันเป๊ะ ต่างแค่ต้นทุน · **ไม่มีจุดไหนที่ตัวเลขติดลบ** แต่ 0.21 ก็คือ "ใช้ไม่ได้จริง" อยู่ดี — และนั่นคือสิ่งที่ทำให้บทนี้อันตรายกว่าที่คิด

25.1 ประเภทของ Transaction Costs

ประเภท	สาเหตุ	ขนาดปกติ (crypto)	สูตร
Bid-Ask Spread	Market maker กำไร	0.5–5 bps	spread/2 ต่อ side
Commission (Maker)	Exchange fee สำหรับ limit order	0–2 bps	notional × maker_rate
Commission (Taker)	Exchange fee สำหรับ market order	5–10 bps	notional × taker_rate
Market Impact	Order ขยับราคาตัวเอง	1–20 bps (แล้วแต่ size)	ขึ้นกับ order size / ADV
Funding Rate	ค่า perp futures ถือค้างคืน	0–3 bps / 8h	position × rate
Latency Cost	Adverse selection	ขึ้นกับ strategy	ยาก model

25.2 Transaction Cost Model ครบถ้วน

```
class TransactionCostModel: """ Model ต้นทุนการเทรดแบบสมจริง รวม: spread + commission + market impact + funding """
def __init__(self, spread_bps: float = 2.0, maker_fee_bps: float = 1.0, taker_fee_bps: float = 7.0,
    impact_coeff: float = 0.1, adv_usd: float = 1_000_000, funding_rate_bps: float = 1.0, funding_period_h: float = 8.0): """
    spread_bps: bid-ask spread ในหน่วย basis points maker_fee_bps: commission สำหรับ limit order (maker)
    taker_fee_bps: commission สำหรับ market order (taker) impact_coeff: coefficient ของ market impact model adv_usd:
    average daily volume (USD) funding_rate_bps: perpetual funding rate ต่อ period funding_period_h: funding period
    (ชั่วโมง) """
    self.spread_bps = spread_bps
    self.maker_fee_bps = maker_fee_bps
    self.taker_fee_bps = taker_fee_bps
    self.impact_coeff = impact_coeff
    self.adv_usd = adv_usd
    self.funding_rate_bps = funding_rate_bps
    self.funding_period_h = funding_period_h

    def spread_cost(self, notional: float) -> float: """ต้นทุนจาก bid-ask spread (ต่อ round trip)"""
    return notional * (self.spread_bps / 10_000)

    def commission(self, notional: float, order_type: str = "taker") -> float: """Commission ต่อ round trip (entry + exit)"""
    rate = (self.taker_fee_bps if order_type == "taker" else self.maker_fee_bps)
    return notional * (rate / 10_000) * 2 # x2: entry + exit

    def market_impact(self, notional: float, volatility: float = 0.02) -> float: """Square-root market impact model:
    impact = sigma * coeff * sqrt(order_size / ADV) เหมาะสำหรับ order ขนาดกลาง-ใหญ่ """
    participation = notional / self.adv_usd
    return notional * volatility * self.impact_coeff * np.sqrt(participation)

    def funding(self, notional: float, hold_hours: float = 24.0) -> float: """ต้นทุน funding สำหรับ perpetual futures"""
    periods = hold_hours / self.funding_period_h
    return notional * (self.funding_rate_bps / 10_000) * periods

    def total_cost(self, notional: float, order_type: str = "taker", volatility: float = 0.02, hold_hours: float = 24.0) -> dict:
    """รวมต้นทุนทั้งหมด"""
    sp = self.spread_cost(notional)
    cm = self.commission(notional, order_type)
    mi = self.market_impact(notional, volatility)
    fn = self.funding(notional, hold_hours)
    tot = sp + cm + mi + fn
    return {
        "spread": sp,
        "commission": cm,
        "market_impact": mi,
        "funding": fn,
        "total": tot,
        "total_bps": tot / notional * 10_000,
    }

    # ตัวอย่าง: เทรด BTC $100,000
    tcm = TransactionCostModel(
        spread_bps=2.0, taker_fee_bps=7.0, impact_coeff=0.10, adv_usd=500_000_000, funding_rate_bps=1.0,
    )
    costs = tcm.total_cost(notional=100_000, order_type="taker", volatility=0.025, hold_hours=24)
    print("Transaction Costs for $100,000 BTC trade:")
    for k, v in costs.items():
        if k == "total_bps":
            print(f"{'TOTAL (bps)':<20}: {v:8.2f} bps")
        else:
            print(f"{k:<20}: ${v:>8.2f}")
```

► OUTPUT

Transaction Costs for \$100,000 BTC trade:

```
-----
spread          : $   20.00
commission       : $  140.00
market_impact    : $    3.54
funding          : $   30.00
total            : $  193.54
TOTAL (bps)      :   19.35 bps
```

25.3 Realistic P&L — "Strategy ดี" หลังหักต้นทุน

📌 **อ่านว่า:** อย่าเพิ่งตกใจกับความยาว — 90 บรรทัดนี้คือ `vectorized_backtest` จากบทที่ 22 แบบคำต่อคำ เพียงแค่เปลี่ยน "หักค่าธรรมเนียมคงที่ตัวเดียว" เป็น "หักต้นทุนจริง 4 ชนิด" . ส่วนที่ *ใหม่จริง ๆ* มีแค่ 2 ก้อน: **ต้นทุนตอนซื้อขาย** (จ่ายเฉพาะแห่งที่เปลี่ยนสถานะ) กับ **ต้นทุนตอนถือ** (funding — จ่ายทุกวันที่ยังถืออยู่) . ที่เหลือข้ามไปได้เลยถ้าอ่านบทที่ 22 มาแล้ว

```
def backtest_with_realistic_costs( price_a: pd.Series, price_b: pd.Series, window: int = 60, entry_z: float =
2.0, exit_z: float = 0.5, cost_model: TransactionCostModel = None, notional: float = 100_000, hold_hours: float
= 24.0, ) -> pd.DataFrame: """ Vectorized backtest พร้อม realistic cost model """ if cost_model is None:
cost_model = TransactionCostModel() df = pd.DataFrame({"A": price_a, "B": price_b}) # Z-score log_spread =
np.log(df["A"]) - np.log(df["B"]) roll_mean = log_spread.rolling(window).mean() roll_std =
log_spread.rolling(window).std() df["zscore"] = (log_spread - roll_mean) / roll_std # Signal + lag raw =
pd.Series(np.nan, index=df.index) raw[df["zscore"] > entry_z] = -1.0 raw[df["zscore"] < -entry_z] = 1.0
raw[df["zscore"].abs() < exit_z] = 0.0 raw = raw.ffmpeg().fillna(0) df["signal"] = raw.shift(1).fillna(0) # Daily
volatility (rolling) daily_vol = log_spread.rolling(window).std() # Gross return ret_a = df["A"].pct_change()
ret_b = df["B"].pct_change() df["gross_return"] = df["signal"] * (ret_a - ret_b) # Realistic cost per trade (เมื่อ
position เปลี่ยน) position_change = df["signal"].diff().abs() cost_per_trade = position_change * (
cost_model.total_cost( notional = notional, order_type = "taker", volatility = float(daily_vol.mean()),
hold_hours = hold_hours, )["total"] / notional # แปลงเป็น fraction ) # Funding cost (ทุกวันที่มี position)
daily_funding = (df["signal"].abs() * cost_model.funding_rate_bps / 10_000 * (24 / cost_model.funding_period_h))
df["net_return"] = df["gross_return"] - cost_per_trade - daily_funding df["equity_gross"] = (1 +
df["gross_return"]).cumprod() df["equity_net"] = (1 + df["net_return"]).cumprod() # ตั้งชื่อ "equity" ให้ตรงกับที่
compute_metrics() คาดหวัง — และเลือกใช้ # equity_net เพราะ metrics ต้องวัดจากเงินหลังหักต้นทุน ไม่ใช่ก่อนหัก df["equity"] =
df["equity_net"] peak_net = df["equity_net"].cummax() df["drawdown"] = (df["equity_net"] - peak_net) / peak_net
return df.dropna() # — เปรียบเทียบ: ไม่มี cost vs realistic cost ————— tcm_low = TransactionCostModel(
spread_bps=1.0, taker_fee_bps=5.0, impact_coeff=0.05, funding_rate_bps=0.5) # optimistic tcm_mid =
TransactionCostModel( spread_bps=2.0, taker_fee_bps=7.0, impact_coeff=0.10, funding_rate_bps=1.0) # realistic
tcm_high = TransactionCostModel( spread_bps=5.0, taker_fee_bps=10.0, impact_coeff=0.20, funding_rate_bps=2.0) #
pessimistic pa_s = pd.Series(bybit, index=dates) pb_s = pd.Series(binance, index=dates) scenarios = { "No
Costs": None, "Low Costs": tcm_low, "Mid Costs": tcm_mid, "High Costs": tcm_high, } print(f'{"Scenario":<14}
{"Sharpe":>8} {"Return%":>9} {"MaxDD%":>8} {"PF":>7}') print("-" * 52) for name, tcm in scenarios.items(): if
tcm is None: res = vectorized_backtest(pa_s, pb_s, window=60, entry_z=2.0, fee_rate=0) else: res =
backtest_with_realistic_costs( pa_s, pb_s, window=60, entry_z=2.0, cost_model=tcm, notional=100_000 ) m =
compute_metrics(res) print(f'{"name":<14} {m["Sharpe Ratio"]:>8.3f} " f"{m["Total Return (%)"]:>9.2f} " f"{m["Max
Drawdown (%)"]:>8.2f} " f"{m["Profit Factor"]:>7.3f}')
```

► OUTPUT

Scenario	Sharpe	Return%	MaxDD%	PF
No Costs	2.669	19.78	-2.24	2.361
Low Costs	1.827	13.42	-2.37	1.761
Mid Costs	1.256	9.22	-2.50	1.468
High Costs	0.209	1.56	-2.84	1.070

🏠 **ต้นทุน 2 ชนิดที่คิดคนละวิธี — และคนมักลืมชนิดที่สอง**

① ต้นทุน "ตอนซื้อขาย" — จ่ายเฉพาะแห่งที่เปลี่ยนสถานะ
`position_change = df["signal"].diff().abs()`

```
cost_per_trade = position_change * (ต้นทุนต่อครั้ง / notional)
```

```
# @ ต้นทุน "ตอนถือ" — จ่ายทุกแห่งที่ยังถืออยู่ ไม่ว่าจะเทรดหรือไม่
```

```
daily_funding = (df["signal"].abs() * - .abs() ไม่ใช่ .diff()
                 funding_rate_bps / 10_000 *
                 (24 / funding_period_h))
```

```
df["net_return"] = df["gross_return"] - cost_per_trade - daily_funding
```

1. ความต่างอยู่ที่ `.diff()` กับ `.abs()` — ต้นทุนซื้อขายผูกกับเหตุการณ์(สถานะเปลี่ยน) จึงใช้ `diff()` · funding ผูกกับสภาพ(ยังถืออยู่ไหม) จึงใช้ `abs()` ของสัญญาณเอง · ใช้ `.abs()` เพราะทั้ง long และ short ก็ต้องจ่าย funding เหมือนกัน
2. ทำไม `funding` ถึงเป็นตัวฆ่ากลยุทธ์เจียบ ๆ — มันไม่ปรากฏในตอนที่คุณกดปุ่ม จึงไม่มีอะไรเตือน · กลยุทธ์ที่ถือยาว ๆ อย่าง pair trade อาจเทรดแค่ 20 ครั้งต่อปี (ต้นทุนซื้อขายน้อยมาก) แต่ถือของอยู่ 200 วัน ซึ่งเป็น 200 วันที่จ่าย funding ทุกวัน · ยิ่งกลยุทธ์เทรตน้อยเท่าไร สัดส่วนของ funding ยิ่งใหญ่ขึ้นเท่านั้น
3. $(24 / \text{funding_period_h})$ — แปลง "อัตราต่อรอบ" เป็น "อัตราต่อวัน" · exchange ส่วนใหญ่คิด funding ทุก 8 ชั่วโมง ดังนั้นตัวคูณคือ 3 · ถ้าลืมคูณตัวนี้จะประเมินต้นทุนต่ำไป 3 เท่า ซึ่งพอมาสะสม 200 วันก็ต่างกันมหาศาล
4. `equity_gross` กับ `equity_net` ถูกเก็บไว้ทั้งคู่ — ไม่ใช่ความสับสนเปลี่ยน · ส่วนต่างระหว่างสองเส้นนี้คือเงินที่จ่ายให้ exchange ทั้งหมด ซึ่งเป็นตัวเลขที่ควรดูตรง ๆ อย่างน้อยครั้งหนึ่ง · หลายคนตกใจว่ามันใหญ่กว่ากำไรสุทธิเสียอีก

ผลกระทบของ TRANSACTION COSTS

จาก Sharpe 2.67 (ไม่มี cost) → 0.21 (high cost) — ต้นทุนกิน edge ไป 92% การเพิ่ม cost ทำให้:

1. Return ลดลง — โดยตรงจากค่าใช้จ่าย (19.78% → 1.56%)
2. Sharpe ลดลงมากกว่า Return — เพราะ cost เพิ่ม variance
3. Max Drawdown แย่ลง — cost ทำให้ drawdown นานขึ้นและลึกขึ้น

⚠️ สังเกตให้ดีว่าไม่มีบรรทัดไหนติดลบเลย — และนั่นคือรูปแบบที่หลอกคนได้มากที่สุด · ถ้าเลขพลิกเป็นลบ ทุกคนจะเห็นทันทีว่าต้องหยุด · แต่ Sharpe 0.21 ยัง "เป็นบวก" ซึ่งชวนให้คิดว่า "ก็ยังพอไหว ลองปรับอีกนิดคงดีขึ้น" · ในความจริง Sharpe 0.21 แปลว่าผลตอบแทนทั้งปีเล็กกว่าความผันผวนหลายเท่า — ปีที่แย่มากปีที่ดีได้สบาย ๆ

🚫 เกณฑ์ที่ควรตั้งไว้ล่วงหน้า (ก่อนเห็นตัวเลข ไม่ใช่หลัง): ถ้า Sharpe หลังต้นทุนจริง ต่ำกว่า 1.0 ให้ถือว่าไม่ผ่าน · ตั้งเกณฑ์ทีหลังเมื่อไร คุณจะขยับมันตามตัวเลขที่ได้เสมอ

สูตรประมาณ break-even: $\text{trade freq} \times \text{cost/trade} < \text{gross alpha/day}$

25.4 Cost Sensitivity Table

```
def cost_sensitivity_table( price_a: pd.Series, price_b: pd.Series, window: int = 60, entry_z: float = 2.0,
                           spread_range: list = None, commission_range: list = None, ) -> pd.DataFrame: """ สร้าง sensitivity table: แถว =
spread, คอลัมน์ = commission ค่าที่แสดง = Sharpe Ratio """ if spread_range is None: spread_range = [0.5, 1.0, 2.0,
3.0, 5.0] if commission_range is None: commission_range = [0.0, 2.0, 5.0, 7.0, 10.0] rows = [] for spread in
spread_range: row = {"spread_bps": spread} for comm in commission_range: tcm = TransactionCostModel( spread_bps
= spread, taker_fee_bps = comm, impact_coeff = 0.10, funding_rate_bps = 1.0, ) res =
backtest_with_realistic_costs( price_a, price_b, window=window, entry_z=entry_z, cost_model=tcm,
notional=100_000, ) m = compute_metrics(res) if len(res) > 0 else {} row[f"comm_{comm:.0f}bps"] = round(
m.get("Sharpe Ratio", 0), 2 ) rows.append(row) return pd.DataFrame(rows).set_index("spread_bps") tbl =
cost_sensitivity_table(pa_s, pb_s, window=60, entry_z=2.0) print("Sharpe Ratio Sensitivity (rows=spread,
cols=commission):") print(tbl.to_string())
```

► OUTPUT

Sharpe Ratio Sensitivity (rows=spread, cols=commission):

	comm_0bps	comm_2bps	comm_5bps	comm_7bps	comm_10bps
spread_bps					
0.5	1.94	1.76	1.50	1.32	1.06
1.0	1.92	1.74	1.48	1.30	1.04

2.0	1.87	1.70	1.43	1.26	1.00
3.0	1.83	1.65	1.39	1.21	0.96
5.0	1.74	1.56	1.30	1.13	0.87

อ่าน Sensitivity Table อย่างไร

เซลล์แต่ละช่องคือ Sharpe Ratio ที่ได้หากใช้ spread + commission ชุดนั้น

- สีเขียว (Sharpe > 1.0): strategy ยังมี edge แม้มี cost
- สีเหลือง (0.5–1.0): marginally profitable
- สีแดง (< 0.5): cost ทำลาย edge หมด

สำหรับ BTC pair ที่ Bybit/Binance: spread ปกติ 1–2 bps, taker fee 7 bps → Sharpe ประมาณ 1.2–1.5 ซึ่งยังดี แต่ถ้าใช้ market order บ่อย (spread 5 bps + comm 10 bps) → Sharpe เหลือ 0.06 ≈ flat

25.5 Market Impact Model — เมื่อ Order ใหญ่กระทบราคา

```
def market_impact_analysis(adv_usd: float = 500_000_000, vol: float = 0.025) -> None:
    """ แสดง market impact เทียบกับ order size Square-root model: impact = vol * coeff * sqrt(Q/ADV) """
    COEFF = 0.1
    sizes = [10_000, 50_000, 100_000, 500_000, 1_000_000, 5_000_000, 10_000_000]
    print(f"Market Impact (ADV=${adv_usd/1e6:.0f}M, vol={vol:.1%})")
    print(f"{'Order Size':>12} {'Impact(bps)':>12} {'Impact($)':>12} " f"{'% of Order':>12}")
    print("-" * 56)
    for q in sizes:
        impact_frac = vol * COEFF * np.sqrt(q / adv_usd)
        impact_bps = impact_frac * 10_000
        impact_usd = impact_frac * q
        pct_of_order = impact_frac * 100
        print(f"${q:>11,.0f} {impact_bps:>12.2f} ${impact_usd:>11,.2f} " f"{pct_of_order:>11.3f}%")
    market_impact_analysis(adv_usd=500_000_000, vol=0.025)
```

► OUTPUT

Market Impact (ADV=\$500M, vol=2.5%)

Order Size	Impact(bps)	Impact(\$)	% of Order

\$ 10,000	0.11	\$ 0.11	0.001%
\$ 50,000	0.25	\$ 1.25	0.003%
\$ 100,000	0.35	\$ 3.54	0.004%
\$ 500,000	0.79	\$ 39.53	0.008%
\$ 1,000,000	1.12	\$ 111.80	0.011%
\$ 5,000,000	2.50	\$ 1,250.00	0.025%
\$ 10,000,000	3.54	\$ 3,535.53	0.035%

Square-Root Market Impact Law

สูตรที่ถูกใช้แพร่หลายที่สุดสำหรับ market impact:

$$\text{impact} = \sigma \cdot \kappa \cdot \sqrt{\frac{Q}{\text{ADV}}}$$

โดย σ = daily vol, κ = impact coefficient (~0.1–0.5), Q = order size, ADV = average daily volume

Key insight: impact ขึ้นตาม *square root* ของ size — trade 4x ใหญ่ขึ้น → impact เพิ่มแค่ 2x

ตัวอย่าง: ประเมินว่า strategy ของเรา Scalable แค่ไหน

```
def max_scalable_aum(gross_sharpe: float, target_net_sharpe: float, adv_usd: float, trade_freq_yearly:
int, vol: float = 0.025, impact_coeff: float = 0.10, fixed_cost_bps: float = 10.0) -> float: """ หา AUM
สูงสุดที่ strategy ยังทำกำไรได้หลังหัก market impact คำนวณ max AUM (USD) """ # gross alpha per trade (ประมาณจาก
Sharpe) alpha_per_trade = gross_sharpe * vol / np.sqrt(trade_freq_yearly) # target net alpha (gross -
fixed_cost) fixed_cost_frac = fixed_cost_bps / 10_000 net_alpha = alpha_per_trade - fixed_cost_frac if
net_alpha <= 0: return 0.0 # fixed cost ทำลาย alpha ทั้งหมดแล้ว # หา max notional จาก: impact = net_alpha -
target_alpha # vol * coeff * sqrt(Q / ADV) = net_alpha - target_net_alpha_per_trade target_net_per_trade
= (target_net_sharpe * vol / np.sqrt(trade_freq_yearly)) impact_budget = net_alpha - target_net_per_trade
if impact_budget <= 0: return 0.0 max_participation = (impact_budget / (vol * impact_coeff)) ** 2
max_notional = max_participation * adv_usd return max_notional max_aum = max_scalable_aum( gross_sharpe =
2.18, target_net_sharpe = 1.0, adv_usd = 500_000_000, trade_freq_yearly = 50, fixed_cost_bps = 10.0, )
print(f"Max scalable AUM: ${max_aum:,.0f} ({max_aum/1e6:.1f}M USD)") print("ใส่ capital เกินนี้ -> Sharpe ต่ำกว่า
1.0 เพราะ market impact")
```

► OUTPUT

```
Max scalable AUM: $804,891,199 (804.9M USD)
ใส่ capital เกินนี้ -> Sharpe ต่ำกว่า 1.0 เพราะ market impact
```

► แบบฝึกหัด: สร้าง cost scenario ที่ "worst case" และ plot equity curves

25.6 สรุป: Checklist ก่อน Live Trading

```
""" Checklist ก่อน deploy strategy ใด ๆ เข้า live trading """ CHECKLIST = { "1. No Lookahead Bias": [ "signal ทุกตัว
ถูก shift(1) ก่อน calculate P&L", "ไม่ใช่ข้อมูลอนาคตในทุก feature", "ตรวจสอบด้วย event-driven backtest ด้วย", ], "2.
Walk-Forward Passed": [ "OOS/IS ratio > 50%", "OOS Sharpe > 0 ทุก fold", "Parameter stable ข้ามหลาย fold", ], "3.
Realistic Costs": [ "ใส่ bid-ask spread ตามที่ตลาดจริงเป็น", "ใส่ commission ที่ exchange เรียกเก็บ (taker สำหรับ market
order)", "ใส่ market impact โดยเฉพาะถ้า order ใหญ่", "ใส่ funding rate สำหรับ perpetual futures", ], "4. Metrics ผ่าน
เกณฑ์ (OOS + Realistic Costs)": [ "Sharpe > 1.0", "Calmar > 1.0", "Max Drawdown < 15%", "Profit Factor > 1.2", ],
"5. Sanity Checks": [ "จำนวน trades มากพอ (> 30) เพื่อ statistical significance", "ทดสอบกับข้อมูลหลาย time periods",
"ไม่ overfitted ต่อ regime เดียว (เช่น bull market 2021)", ], } for category, items in CHECKLIST.items():
print(f"\n{category}") for item in items: print(f" ☐ {item}") print("\nถ้าผ่านทุกข้อ -> เริ่ม paper trading 1-3 เดือน
ก่อน live")
```

► OUTPUT

1. No Lookahead Bias
 - ☐ signal ทุกตัวถูก shift(1) ก่อน calculate P&L
 - ☐ ไม่ใช่ข้อมูลอนาคตในทุก feature
 - ☐ ตรวจสอบด้วย event-driven backtest ด้วย
2. Walk-Forward Passed
 - ☐ OOS/IS ratio > 50%
 - ☐ OOS Sharpe > 0 ทุก fold
 - ☐ Parameter stable ข้ามหลาย fold
3. Realistic Costs
 - ☐ ใส่ bid-ask spread ตามที่ตลาดจริงเป็น
 - ☐ ใส่ commission ที่ exchange เรียกเก็บ (taker สำหรับ market order)
 - ☐ ใส่ market impact โดยเฉพาะถ้า order ใหญ่
 - ☐ ใส่ funding rate สำหรับ perpetual futures
4. Metrics ผ่านเกณฑ์ (OOS + Realistic Costs)
 - ☐ Sharpe > 1.0
 - ☐ Calmar > 1.0
 - ☐ Max Drawdown < 15%
 - ☐ Profit Factor > 1.2
5. Sanity Checks
 - ☐ จำนวน trades มากพอ (> 30) เพื่อ statistical significance

- ทดสอบกับข้อมูลหลาย time periods
 - ไม่ overfitted ต่อ regime เดียว (เช่น bull market 2021)
- ถ้าผ่านทุกข้อ – เริ่ม paper trading 1-3 เดือนก่อน live

► แบบฝึกหัด: คำนวณ minimum edge ที่ต้องมีเพื่อ breakeven หลัง cost

จบ Part IV — สิ่งที่ได้จาก Backtesting

ก่อน Part IV: เทรน model ดูผลดี แต่ไม่รู้ว่าเป็น edge จริงหรือแค่ lucky/overfit

หลัง Part IV: มีกระบวนการที่เชื่อถือได้ก่อน deploy ทุกครั้ง

- บท 22 — Vectorized backtest เร็วสำหรับ parameter sweep พร้อม shift(1) หยุด lookahead bias ✓
- บท 23 — Event-driven backtest สมจริง จำลอง fill และ slippage ทีละ bar ✓
- บท 24 — Walk-forward validation เผย OOS/IS ratio และ parameter stability ✓
- บท 25 — Realistic cost model (spread + commission + impact + funding) พร้อม sensitivity table ✓

ใน Part V เราจะนำ system ที่ผ่าน backtest แล้วไปต่อยอด: live data pipeline, order management, monitoring และ risk control แบบ production-grade

■ [สารบัญทั้งหมด](#) · [คำศัพท์ Quant Trading](#)

← Part III: OOP

Part V: AI-Assisted Coding →